

BPC-PRP Project Documentation

1st Miroslav Bařista

Brno University of Technology, Faculty of Information Technology
Brno, Czech Republic
xbasism00@vutbr.cz

2nd řimon Dufek

Brno University of Technology, FIT
Brno, Czech Republic
xdufeks00@vutbr.cz

Abstract—This paper presents an autonomous differential-drive robot implementation for the BPC-PRP [1] final exam maze escape task. The robot navigates an 8×8 grid of 0.40 m cells bounded by black tape, using analog IR sensors (TCRT5000) for line detection, a monocular camera for ArUco tag recognition, a 2D LiDAR (RPLIDAR A1) for wall proximity estimation. Line position is estimated via differential analog readings with calibration and linearization, while a discrete PID controller computes angular velocity commands to maintain trajectory tracking at constant forward speed. Inverse differential kinematics translates control signals into individual wheel velocities. A finite-state machine manages high-level behaviors: straight following, intersection detection, tag-based decision making. In maze you can collect minotaurs (penalty: +30 s) or treasure (bonus: −30 s). The final exam show successful maze escape in 02:04 s with 0 minotaur penalties and treasure collected, demonstrating robust integration of multi-sensor perception, real-time control, and reactive navigation under competition constraints. The approach provides a practical educational framework for combining low-level actuation with high-level task planning in constrained, partially observable environments.

Index Terms—Robotics, project, BPC-PRP, state-of-the-art

I. INTRODUCTION

Autonomous mobile robots are increasingly deployed in structured indoor environments such as warehouses, hospitals, and factories, where reliable path tracking and real-time decision-making are critical. The BPC-PRP final exam task simulates these challenges by requiring a robot to navigate an unknown maze and interpret ArUco markers to optimize its path [3]. The task integrates core robotics competencies, including sensor perception, feedback control, kinematic modeling, and reactive navigation.

Navigating such a constrained environment introduces several technical challenges. First, maintaining accurate line estimation is difficult due to surface reflections, ambient light variations, and sensor saturation during sharp turns. Second, the 2D LiDAR exhibits a blind zone at distances below approximately 15 cm, which complicates proximity-based navigation, particularly because maze walls are not perfectly aligned across all cells. Third, ArUco marker detection at intersections requires precise temporal synchronization. Tag readings at the current junction must be carefully decoupled to prevent misinterpretation of navigation instructions for subsequent cells. Finally, IMU-assisted rotational maneuvers are prone to overshoot and gradual yaw drift, while wheel slip, mechanical backlash, and motor deadband introduce additional trajectory deviations during turning.

To address these challenges, this paper presents a modular control architecture combining low-level perception, continuous feedback control, and rule-based navigation. The maze-solving robot operates within a finite state machine loop, where each state corresponds to a specific navigation behavior and environmental context.

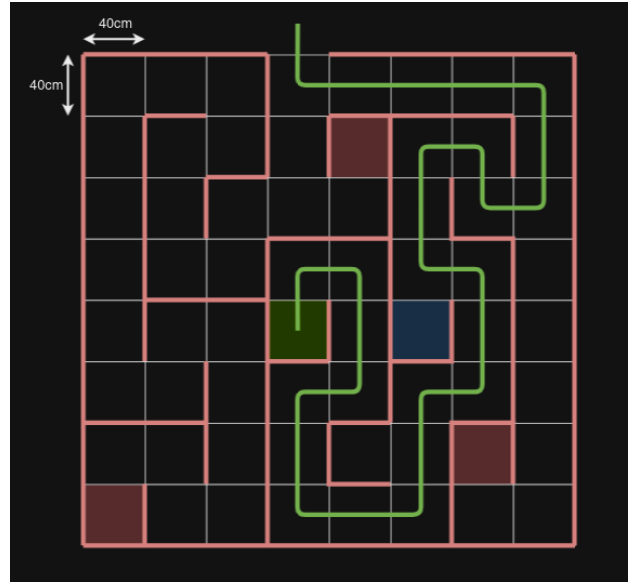


Fig. 1. Maze example

II. MAIN TEXT

A. Software Architecture and Perception Layer

The navigation system is implemented in C++ on the ROS2 Humble framework, utilizing a modular node-based architecture. Each functional component operates as an independent ROS2 node, enabling asynchronous communication, clear separation of perception and control, and fault isolation.

Camera Node: The camera node subscribes to the `/bpc_prp_robot/camera/compressed` topic and acquires compressed image frames from the onboard camera. The received frames are decoded and processed using the OpenCV ArUco module to detect and decode fiducial markers. Detected marker identifiers are subsequently published on the `/bpc_prp_robot/arucos_detected` topic.

IMU Node: The IMU node subscribes to the `/bpc_prp_robot/imu` topic and extracts the angular

velocity about the vertical (z) axis. During system startup, the node performs a static calibration procedure while the robot remains stationary in order to compensate for sensor bias and drift. After successful calibration convergence, the node publishes a readiness signal on `/bpc_prp_robot/imu_ready` and continuously provides estimated yaw values on the `/bpc_prp_robot/yaw` topic.

LiDAR Node: The LiDAR node processes raw range measurements obtained from the `/bpc_prp_robot/lidar` topic. The acquired 360° scan is partitioned into four angular sectors of interest, namely front-left, front, front-right, left, and right, corresponding to regions most relevant for maze navigation and obstacle avoidance. To reduce the influence of measurement noise and outliers, a median filtering technique is applied independently to each sector. The filtered distance estimates are published on the `/bpc_prp_robot/side_distances` and `/bpc_prp_robot/front_distance` topics.

Line Node: The line node subscribes to the `/bpc_prp_robot/line_sensors` topic, which provides raw reflectance sensor measurements within the range of 0–1024, where lower values correspond to white surfaces and higher values correspond to black surfaces. The node evaluates the incoming sensor values against a predefined threshold and publishes a binary detection signal on `/bpc_prp_robot/line_detected` whenever a line is detected by at least one sensor.

Motor Node: The motor node serves as a centralized interface for motor actuation. It subscribes to high-level control commands from the `/bpc_prp_robot/motor_commands` topic and publishes the resulting motor speed commands to `/bpc_prp_robot/set_motor_speeds`. Although direct motor control from individual nodes would be possible, the centralized architecture improves modularity, maintainability, and coordination between subsystems.

Maze Loop Node: The maze loop node coordinates communication between the previously described nodes by subscribing to and publishing the relevant topics. To ensure synchronization and consistent data exchange between individual nodes, the control loop operates with a period of 20 ms. The node implements the primary navigation logic using a finite-state machine architecture.

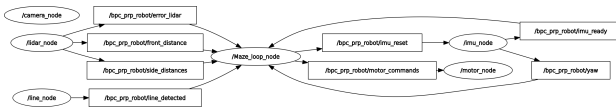


Fig. 2. Rqt_graph of nodes and topics

B. Maze solver algorithm

At system startup, the control state is initialized to CALIBRATION, during which the algorithm waits for the `is_imu_ready_` signal indicating successful IMU calibration. Once the calibration process is completed, the state machine transitions to the CORRIDOR_FOLLOWING state and

initializes `target_yaw_` to maintain straight-line motion within the corridor.

Corridor following is based on two complementary mechanisms. The first mechanism evaluates wall validity and proximity using LiDAR measurements. If one of the walls is detected to be excessively close to the robot, the state machine transitions to the ONE_WALL_FOLLOWING state, where a virtual lateral error is recalculated, assuming a nominal corridor width of 20 cm. The second mechanism monitors the opposite wall for discontinuities or openings. If an opening is detected on one side, the controller switches to ONE_WALL_FOLLOWING state and starts following the continuous wall on the opposite side in order to maintain stable navigation and avoid entering unintended openings.

The lateral control error is computed using a LiDAR-based PID controller with parameters `pid_controller(12.5f, 0.5f, 2.0f)`. In addition, heading stabilization is achieved using IMU-based yaw correction with a proportional gain of $P = 3$.

In the ONE_WALL_FOLLOWING state, the algorithm continuously monitors the presence of the missing wall. Once the previously missing wall is detected again, the state machine transitions back to the CORRIDOR_FOLLOWING state.

Simultaneously, the controller evaluates whether the currently active ArUco marker indicates a navigation direction corresponding to an opening in the wall. If such a condition is satisfied, the state machine transitions to the DRIVE_TO_CENTER state, in which the robot moves toward the center of the intersection before initiating the turning maneuver.

In addition, if an obstacle or wall is detected directly in front of the robot, the controller transitions to the TURNING state in order to perform a turn toward the available opening.

At the final stage of the control process, a corrective control action is computed using the same LiDAR-based PID controller. However, the proportional gain of the IMU-based heading correction is reduced to $P = 0.8$. This reduction improves stability during one-wall following, as this navigation mode typically occurs near intersections and turns, where excessively high proportional gains could destabilize the robot trajectory.

The DRIVE_TO_CENTER state serves as a transitional delay stage that allows the robot to move toward the geometric center of the intersection before initiating the turning maneuver. This approach prevents premature rotation immediately after detecting an opening and improves turning accuracy and overall trajectory stability within the intersection area.

In the INTERSECTION_STRAIGHT state, the robot relies exclusively on IMU-based heading control with a proportional gain of $P = 4.5$ to maintain a straight trajectory while traversing the intersection.

During this state, the algorithm evaluates whether the robot has reached a T-shaped intersection by checking for the presence of a wall directly in front of the robot using LiDAR measurements. If a frontal wall is detected, the controller checks for an active ArUco marker specifying the desired

navigation direction. In the absence of a valid marker, the robot follows a default fallback strategy and performs a right turn.

Additional fallback mechanisms are implemented to improve robustness. If the line sensors detect a line crossing, the controller assumes that the robot has successfully exited the intersection area. Similarly, if the LiDAR sensor detects surrounding corridor walls, the state machine transitions back to the `CORRIDOR_FOLLOWING` state.

In the `TURNING` state, the control error is computed as the difference between the target yaw angle and the current yaw measurement obtained from the IMU. The resulting angular error is normalized to ensure continuity within the range of valid orientation values.

This state represents the only stage in which an IMU-based PID controller is actively used for rotational control, defined as `pid_controller_imu(10.0f, 2.5f, 2.0f)`. To prevent excessive rotational speed and improve stability, the angular error is clamped to a maximum absolute value of 15.0 degrees.

During execution, the robot continuously reduces the yaw error as it approaches the desired orientation. Once the absolute error falls below the defined threshold of 0.03 radians, the turning motion is considered complete, and the state machine transitions to the `EXIT_CORNER` state.

In the `EXIT_CORNER` state, the robot performs straight-line motion regulated exclusively by IMU-based heading control with a proportional gain of $P = 4.0$. The system maintains the desired orientation while exiting the intersection area.

The transition from this state is primarily triggered by the detection of a line crossing using the line sensor array. Additionally, a fallback mechanism is implemented to ensure robustness: if no line is detected for a continuous duration of 3 seconds, the system assumes that the robot has successfully exited the corner region. In both cases, it is a transition to `CORRIDOR_FOLLOWING` state.

III. CONCLUSION

This project provided valuable practical experience in the design and implementation of autonomous robotic systems using the ROS 2 framework [2]. A major contribution of the work was the development of a robust navigation solution capable of operating in partially observable and non-deterministic environments, where sensor uncertainty, mechanical imperfections, and timing inconsistencies must be continuously considered during system design.

The project significantly improved understanding of multi-sensor integration and real-time robotic control, including practical work with IMU sensors, monocular vision systems, LiDAR-based perception, wheel encoders, and infrared line sensors. In addition to software development, the implementation process emphasized the importance of accounting for hardware-related limitations such as sensor noise, wheel slip, mechanical backlash, calibration inaccuracies, and actuator nonlinearity.

ACKNOWLEDGMENT

The authors gratefully acknowledge the guidance and support provided by the BPC-PRP course instructors and teaching assistants at Brno University of Technology. We thank them for the comprehensive lecture materials, as well as for access to the laboratory and robotic platforms used in this project.

REFERENCES

- [1] Adam Ligocki, Petr Šopák, and Jakub Minařík. *BPC-PRP - Practical Robotics and Computer Vision*. 2025. URL: <https://robotics-but.github.io/BPC-PRP> (visited on 05/14/2026).
- [2] Open Robotics. *ROS 2 Humble Documentation*. 2026. URL: <https://docs.ros.org/en/humble/index.html> (visited on 05/14/2026).
- [3] OpenCV. *Detection of ArUco Markers*. 2026. URL: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html (visited on 05/14/2026).

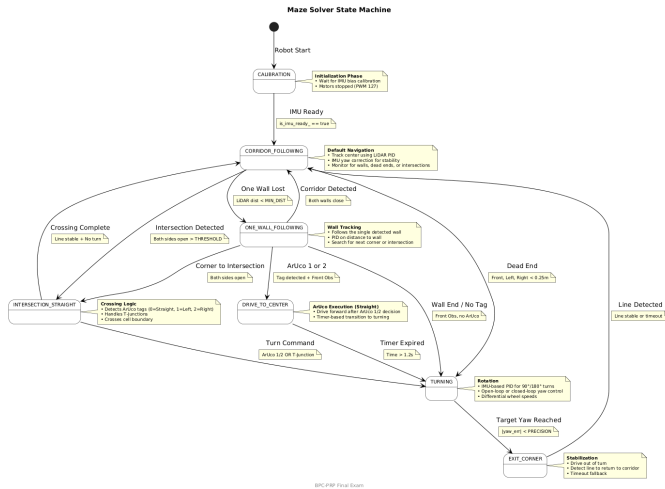


Fig. 3. Maze solver state machine